

编程题

一. 龙格-库塔公式

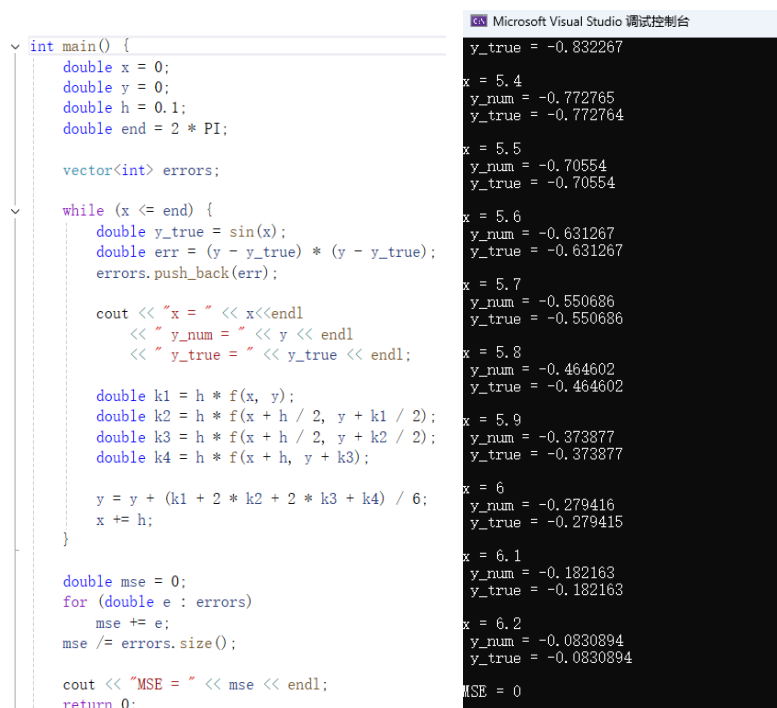
(a) 核心算法大致如下:

```
for(int i = 0; i < N; i++) {  
    k1 = h * f(x, y);  
    k2 = h * f(x + h/2, y + k1/2);  
    k3 = h * f(x + h/2, y + k2/2);  
    k4 = h * f(x + h, y + k3);  
  
    y = y + (k1 + 2*k2 + 2*k3 + k4)/6;  
    x += h;  
}
```

设区间被划分为 N 个步长, 则算法需要进行 N 次迭代。在每次迭代中, 仅进行常数级计算 (计算四个中间变量 k_1, k_2, k_3, k_4), 因此总时间复杂度为 $O(N)$; 在算法执行过程中, 仅需存储当前的变量, 所需空间为常数级。如果需要存储所有计算节点的结果用于后续分析, 则空间复杂度为 $O(N)$ 。

(b) 采用龙格-库塔方法对微分方程 $y' = \cos x$ 进行数值求解, 初值为 $y(0)=0$ 。在区间 $[0, 2\pi]$ 上以步长 $h=0.1$ 进行迭代计算, 通过逐步更新函数值得到各节点处的数值解。同时将数值解与真实解 $y = \sin x$ 进行比较, 用公式计算各节点的误差, 并最终求出均方误差以评估算法精度。

核心代码和测试案例截图如下。MSE 结果为 0 是因为误差较小, 即使将 precision 设为 12 位也是 0。



```
int main() {  
    double x = 0;  
    double y = 0;  
    double h = 0.1;  
    double end = 2 * PI;  
  
    vector<int> errors;  
  
    while (x <= end) {  
        double y_true = sin(x);  
        double err = (y - y_true) * (y - y_true);  
        errors.push_back(err);  
  
        cout << "x = " << x << endl  
              << "y_num = " << y << endl  
              << "y_true = " << y_true << endl;  
  
        double k1 = h * f(x, y);  
        double k2 = h * f(x + h / 2, y + k1 / 2);  
        double k3 = h * f(x + h / 2, y + k2 / 2);  
        double k4 = h * f(x + h, y + k3);  
  
        y = y + (k1 + 2 * k2 + 2 * k3 + k4) / 6;  
        x += h;  
    }  
  
    double mse = 0;  
    for (double e : errors)  
        mse += e;  
    mse /= errors.size();  
  
    cout << "MSE = " << mse << endl;  
    return 0;  
}
```

```
y_true = -0.832267  
x = 5.4  
y_num = -0.772765  
y_true = -0.772764  
x = 5.5  
y_num = -0.70554  
y_true = -0.70554  
x = 5.6  
y_num = -0.631267  
y_true = -0.631267  
x = 5.7  
y_num = -0.550686  
y_true = -0.550686  
x = 5.8  
y_num = -0.464602  
y_true = -0.464602  
x = 5.9  
y_num = -0.373877  
y_true = -0.373877  
x = 6  
y_num = -0.279416  
y_true = -0.279415  
x = 6.1  
y_num = -0.182163  
y_true = -0.182163  
x = 6.2  
y_num = -0.0830894  
y_true = -0.0830894  
MSE = 0
```

二．求解水面高度

2. 想象你有一个底部高低不平的水池，水池底部有 n 个坑（或区域），每个坑的高度由 α_i 决定（每个区域底面积为 1）。如果 α_i 很大，说明这个坑很浅；如果 α_i 很小，说明这个坑很深。你有一桶水，总量为 M ，当你把水倒进去后，由于重力作用，水会先填满最深的坑。最终，所有有水的坑都会达到一个**统一的水面高度**，请你设计一个时间复杂度尽可能低的算法来求解水面的高度，并编程实现，你能数学证明你的算法是最优的吗？在文档中给出当 $\{\alpha_i\} = \{0.2, 0.5, 0.1, 0.8, 1.5, 0.4, 0.9\}$ 且 $M = 2$ 时你的算法结果，并自己再举出一个测试用例并验证。

首先对抗的高度从小到大进行排序，然后从最低高度开始逐层填充水量。在每一层中，将当前层提升到下一层的水量。然后检查剩余水量，如果水量足够，则继续向上填充；否则根据剩余水量计算最终水面高度。时间复杂度 $O(n \log n)$ ，空间复杂度 $O(1)$ 。

数学证明：假设存在一种更优方案，使得某个较高的坑 j 被填充，而较低的坑 i ($\alpha_i < \alpha_j$) 仍未填满。那么可以将一部分水从坑 j 转移到坑 i ，使得整体水面更加均匀，同时不会增加总用水量。因此原方案不是最优的，产生矛盾。

核心代码和测试案例一的结果大致如下：

```
6 double solve(vector<double>& a, double M) {
7     int n = a.size();
8     sort(a.begin(), a.end()); //排序
9
10    for(int i=1;i<n;i++){
11        double diff=a[i]-a[i-1];
12        double needed=diff*i; //需要的水量
13
14        if(needed<=M){
15            M-=needed; //扣除水量
16        }else{
17            return a[i-1]+M/i; //水不够，计算最终水位
18        }
19    }
20
21    return a[n-1]+M/n; //所有水都用完，计算最终水位
22 }
23
24 }
```

Final water height: 0.8

测试案例二为：

```
int main() {
    vector<double> a = {1, 2, 3};
    double M = 3;

    double result = solve(a, M);

    cout << "Final water height: " << result << endl;

    return 0;
}
```

结果如下：

Final water height: 3